Module UART: tóm tắt

Bộ tăng tốc TinyML trên FPGA (Basys 3 / Nexys A7-100T) · file `uart_rx.sv`, `uart_tx.sv`

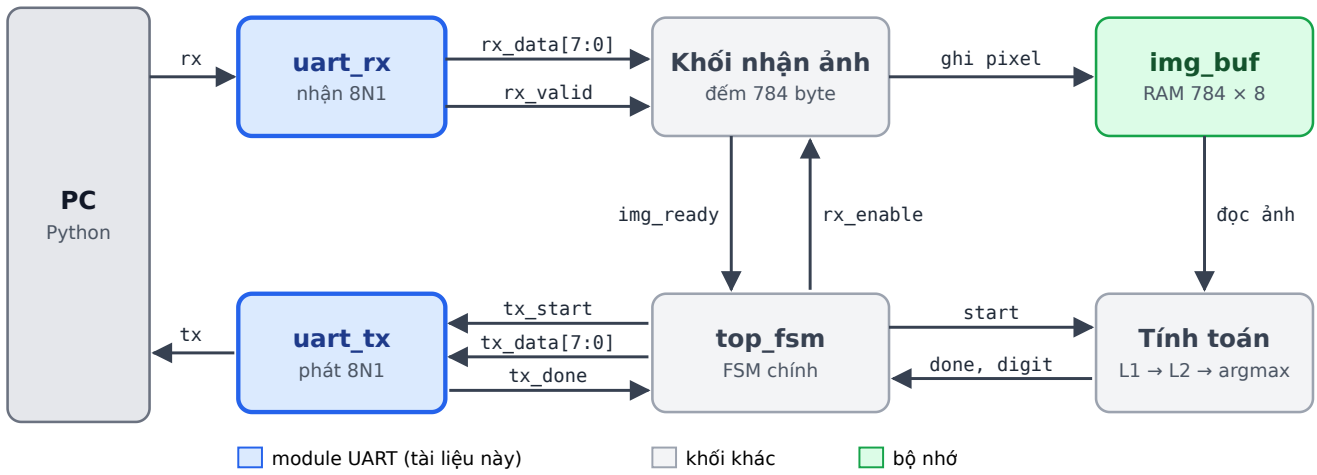
1. UART dùng để làm gì?

UART là đường nói chuyện giữa **PC và FPGA** qua cáp USB của board. Trong đề tài nó chỉ làm 2 việc:

Module	Chiều	Việc làm
`uart_rx`	PC → FPGA	Nhận **ảnh** 28×28 = **784 byte** (mỗi pixel 1 byte, 0–255), ghép từng bit lại thành byte rồi đưa cho khối nhận ảnh.
`uart_tx`	FPGA → PC	Gửi **1 ký tự** kết quả `'0'`–`'9'` (chữ số mạng nhận ra).

> **Lưu ý:** trọng số mạng (`w1/w2/b1/b2.mem`) **không** đi qua UART. Vivado nạp sẵn chúng vào BRAM lúc tổng hợp. UART chỉ chở ảnh vào và kết quả ra.

2. Sơ đồ khối và kết nối



Tín hiệu	Từ → Đến	Ý nghĩa
`rx`	PC → uart_rx	Dây nối tiếp, từng bit của ảnh
`rx_data, rx_valid`	uart_rx → Khối nhận ảnh	Mỗi lần có 1 byte (1 pixel) mới
`rx_enable`	top_fsm → Khối nhận ảnh	Chỉ cho ghi ảnh mới khi FSM đang rảnh
`img_ready`	Khối nhận ảnh → top_fsm	Đã đủ 784 byte, bắt đầu tính
`tx_start, tx_data`	top_fsm → uart_tx	Gửi ký tự kết quả '0'–'9'
`tx_done`	uart_tx → top_fsm	Gửi xong, FSM quay về chờ ảnh mới
`tx`	uart_tx → PC	Dây nối tiếp, ký tự kết quả

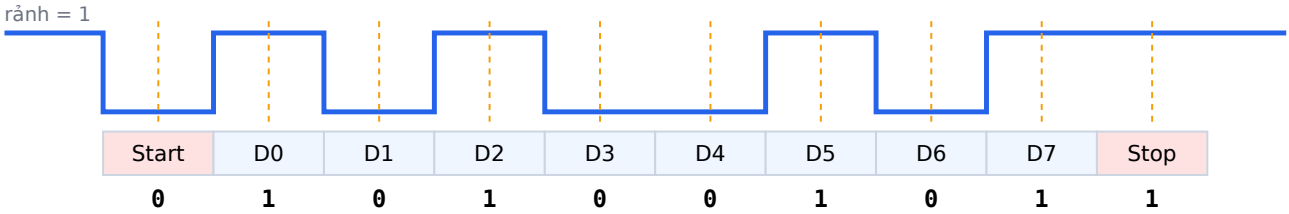
Board	Chân RX (PC → FPGA)	Chân TX (FPGA → PC)
Basys 3	B18	A18
Nexys A7-100T	C4	D4

Chân lấy theo master XDC của Digilent; nên đối chiếu lại khi viết file .xdc.

Không cần module bọc gộp 2 khối: chúng độc lập và nối vào 2 chỗ khác nhau, nên gọi thẳng trong file top của board.

3. Khung truyền 8N1

Thông số chung: **1.000.000 baud (1 Mbaud), 8 bit dữ liệu, không parity, 1 bit stop**. Clock 100 MHz nên mỗi bit dài `100.000.000 / 1.000.000 = 100` chu kỳ (chia hết, không sai số baud) (lấy từ `CLKS_PER_BIT` trong `tinyml_pkg`). Mỗi byte gồm 10 bit, bit thấp (LSB) gửi trước:



Ví dụ gửi byte 0xA5 · mỗi ô = 1 bit = 100 chu kỳ = 1 µs · đường nét đứt cam = điểm lấy mẫu (giữa bit)

Một ảnh 784 byte × 10 bit × 1 µs ≈ **7,8 ms**. Phần tính toán của mạng chỉ mất khoảng 32 µs, nên UART là phần chậm nhất của cả hệ thống.

4. uart_rx: bộ nhận

Cổng	Hướng	Ý nghĩa
`clk, rst`	vào	Clock 100 MHz, reset mức 1
`rx`	vào	Chân RX từ chip USB-UART
`rx_data[7:0]`	ra	Byte vừa nhận
`rx_valid`	ra	Xung 1 chu kỳ: có byte mới
`rx_error`	ra	Xung 1 chu kỳ: bit stop sai

Trạng thái	Làm gì
`S_IDLE`	Chờ rx xuống 0 (bit start)
`S_START`	Đợi nửa bit, kiểm tra rx vẫn = 0; nếu không thì là nhiễu, quay lại
`S_DATA`	Mỗi 100 chu kỳ lấy 1 bit, đủ 8 bit
`S_STOP`	rx phải = 1 → `rx_valid`, sai → `rx_error`

Hai điểm thiết kế quan trọng:

- **Lấy mẫu ở giữa bit** (đợi nửa bit sau cạnh start). Nhờ vậy vẫn nhận đúng khi baud của PC lệch vài phần trăm.
- **Đồng bộ 2 flip-flop** cho chân `rx`, vì tín hiệu đến từ bên ngoài, không cùng clock với FPGA. Bỏ bước này có thể gây lỗi chập chờn (metastability).

5. uart_tx: bộ phát

Cổng	Hướng	Ý nghĩa
tx_start	vào	Xung: bắt đầu gửi
tx_data[7:0]	vào	Byte cần gửi
tx	ra	Chân TX tới PC
tx_busy	ra	1 = đang gửi
tx_done	ra	Xung 1 chu kỳ: gửi xong

Trạng thái	Làm gì
S_IDLE	tx = 1; có `tx_start` thì chốt dữ liệu
S_START	Phát bit 0 trong 1 bit
S_DATA	Phát 8 bit, LSB trước
S_STOP	Phát bit 1, xong thì báo `tx_done`

`tx_start` đến khi đang bận sẽ bị bỏ qua, nên không bao giờ làm hỏng byte đang gửi.

6. Kiểm chứng bằng mô phỏng (tb_uart.sv)

Khi mô phỏng không có PC thật, testbench đóng vai PC bằng task `pc_send`, tự đảo chân rx theo từng bit.

Bài kiểm tra	Kết quả
Nối vòng tx → rx, gửi đủ 256 giá trị 0x00–0xFF	Nhận đúng 256/256, có đủ 256 xung `tx_done`
PC gửi với baud lệch −3 %, 0 %, +3 %	Nhận đúng tất cả
Cố tình làm sai bit stop	Báo `rx_error` đúng
Gửi `tx_start` khi đang bận	Bị bỏ qua đúng
Lint Verilator `-Wall`	Không cảnh báo

Chạy trong Vivado: thêm `tinyml_pkg.sv`, `uart_rx.sv`, `uart_tx.sv` vào Design Sources và `tb_uart.sv` vào Simulation Sources → chọn `tb_uart` làm top → Run Behavioral Simulation → gõ `run all` → Tcl Console in `PASS uart`.

> **Muốn đổi tốc độ truyền** (hiện là 1 Mbaud; phía PC mở cổng với `baudrate=1000000`): chỉ sửa `UART_BAUD` trong `tinyml_pkg.sv` và tốc độ trong `send_image.py`. Không cần sửa code UART.